Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

VLSI DESIGN METHODOLOGIES

BY MAVEN SILICON

Submitted for the course of

ECE1902-INDUSTRIAL INTERNSHIP

In

Electronics and communication Engineering

by:

Sourodipto

Bhowmick

19BEC0803

CONTENTS

Sr No.	Title	Page No.
1	Declaration	3
2	Certificate	4
3	Acknowledgement	5
4	Introduction	6
5	VLSI DESIGN FLOW	7
6	Logic Circuits and Boolean Algebra	10
7	Combinational Logic Circuits	15
8	Sequential Logic Circuits	18
9	Finite State Machines	21
10	Glitches and Hazards	23
11	Memories	24
12	Verilog HDL	25
13	Conclusion	26
14	Future Scope	26
15	References	27

DECLARATION

I hereby declare that the Industrial Internship report entitled "VLSI Design Methodologies" submitted by me to Vellore Institute of Technology, Vellore in partial fulfilment of the requirement for the award of the degree of Bachelor of Technology in Electronics and Communication Engineering. I further declare that the work reported in this report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: VIT, Vellore

Date: 28 Mar 2022

Name: Sourodipto Bhowmick

Reg. No.: 19BEC0803

CERTIFICATION

ACKNOWLEDGEMENT

I hereby acknowledge that I have reviewed the recorded presentation, reviewed information on the informational website and received information on the course "VLSI Design Methodologies" in its entirety. I agree to abide by the principles that were explained in this training. I understand that if I have any questions about the training, materials presented or information not addressed in the training, or if I encounter any problems, it is my responsibility to seek clarification from the designated Human Resources Liaison and/or Human Resources.

INTODUCTION

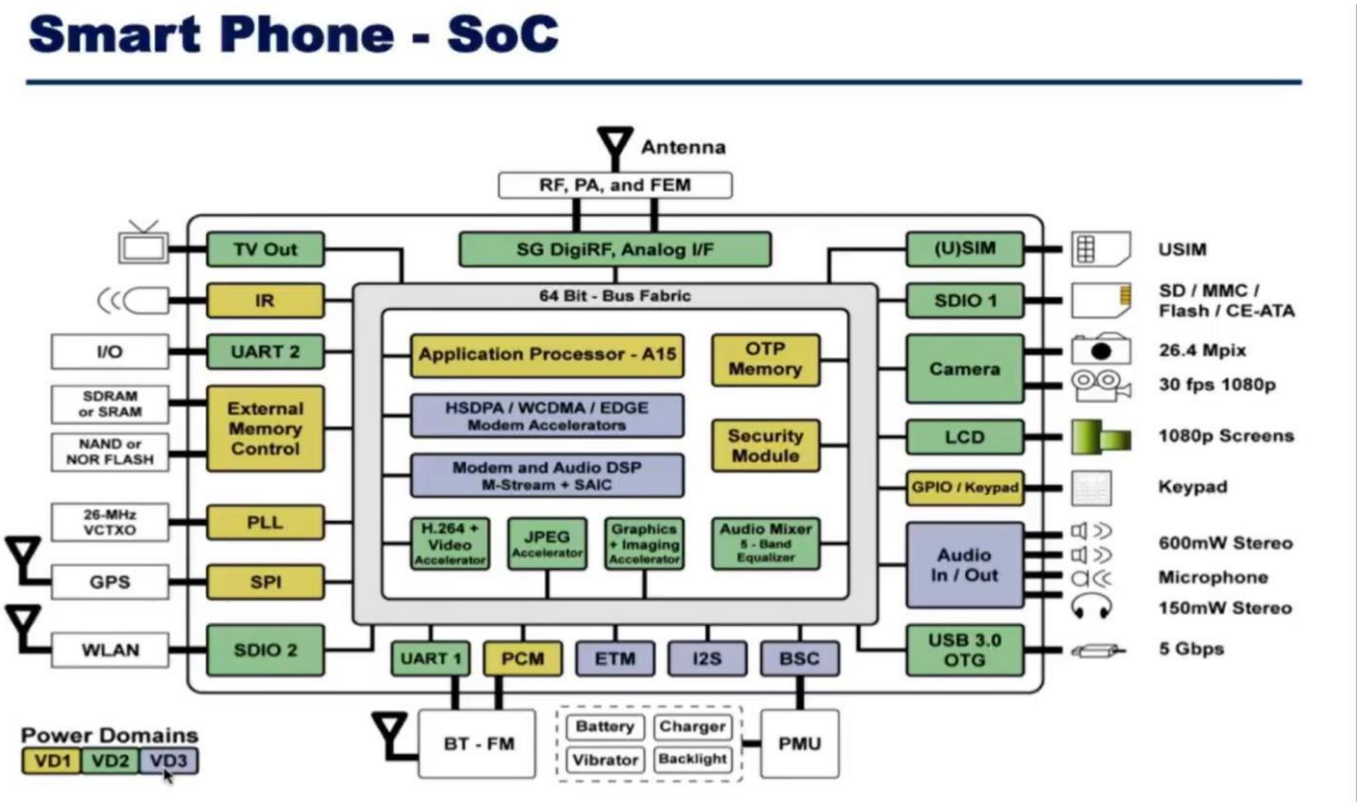
Very large-scale integration or VLSI is a process in which millions of MOS transistors are combined and integrated on a single semiconductor microchip. With the global semiconductor revenue crossing USD 440 Billion in 2020, there is an increasing need to design and produce highly efficient and specialized chips that can power new age technologies such as AI/ML, IoT, AR/VR, Cloud etc., which are increasingly becoming mainstream instead of remaining niche technologies.

Growth in consumer electronics, computing devices, post pandemic, smartphones, intelligent vehicles etc., has further increased the demand.

Designing of a complex SOC:
Take an Iphone for example, a central aspect behind its working is the chip called A11, which is referred to as the SOC-System on Chip. It is complex which constitutes over 4 billion transistors. An SOC in general, has a bus called On-chip bus through which all the IPs(intellectual property) are connected. The centre of it consists of an engine, which processes the information. The engine consists of various types of processor IPs. Various kinds of interfaces (I/Ps and O/Ps) are required to be connected with the processor. Thus, we have to connect different types of peripherals through interface IPs.

For example, in the case of mobile phone, it is needed to connect the SIM with the engine to establish the interface, SIM. Similarly, there are dedicated interface units for camera, audio, display, keypad, etc.

This is why, the chip is composed of processor IP and interface IP, and thus it requires so many IPs and hence, the complexity.The common occurrence in real time companies like Samsung or Apple, is that they don’t manufacture the entire SOC on their own. Rather they buy different IPs of the whole, from different vendors who are specialized in its designing of that specific IP.

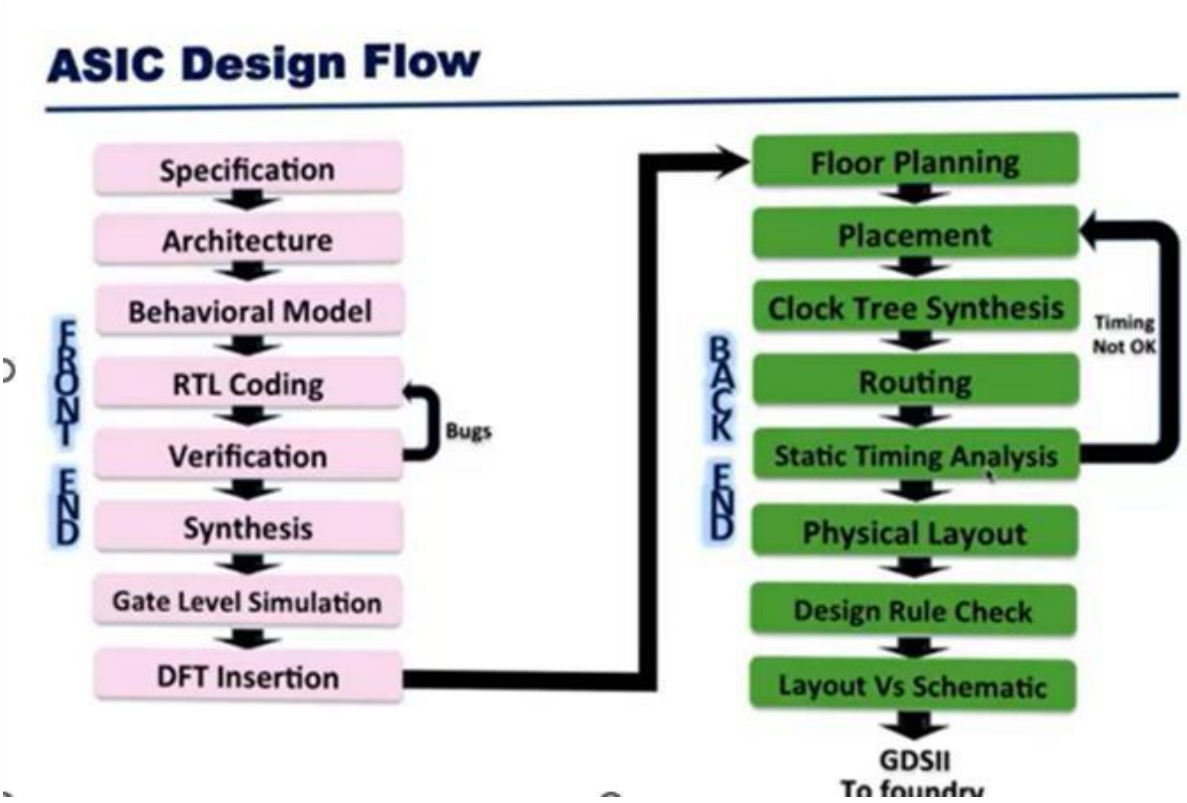


In the above example, we can observe that, based on the activity of block we differentiate the blocks, based on the blocks that require high performance levels to ones which require low, and we reduce voltage accordingly. The PMU(Power Management Unit) works on regulating the power, based on the requirements at that particular instance.

VLSI Design Flow:

ASIC design flow is recommended for complex chips, and
FPGA, or even standard design methods are used for simpler chips. Design flow depends on various factors like interface, kind of clock, packet structure, methods to drive the packets into the chip, etc.(conversion to binary because it only understands it).In VLSI, protocol is very important when we aim to build complex SOCs to connect various IPs, and hence, we follow standard protocols like AHB(Advanced High-Performance Bus), AXI(Advanced Extensible Interface),APB(Advanced Peripheral Bus) to get into a certain VLSI industry.

In the given course, we explore and understand architecture, behavioral model, RTL coding, verification, and synthesis, as part of this online VLSI methodologies course.



We start with the specifications, and based on the functionality we create an architecture. And based on the architecture, we create a behavioral model(to generate the expected values for verification). Based on the architecture, we create a register transfer level coding model, which is a synthesizable model.(We use HDL to create RTL). We simulate the RTL and verify the functionality, and in order to do so, we write a test bench. We simulate the RTL and verify the functionality. And after verification, we synthesize the RTL and we generate the net list.

And after generating the net list, we insert the DFT logic in the net list. So, basically we replace the regular flip flops by scan flip flops.

After that we decide on the floor planning on how the components can be placed efficiently based on the shape of the chip. And after placement we do the clock tree synthesis, mainly aiming on how to avoid clock skew. And then we do the routing and the connections. And only after the connections, we come to know about wire delay.

So we go for static time analysis to make sure thar our timing is perfect.

And then we create the physical layout, on how the actual chip is going to be and how the transistors are going to be realized on actual silicon.So, we create the layout and check whether the physical layer follows the standards and meet its guidelines of fabrication. And finally we cross check the layout with the schematic whether the routed netlist retains the same functionality, and after the physical verification, we release the netlist to GDS II

foundry.
.(To put it in simple words, in front end we stress on verification and in the back end we focus on timing.

LOGIC CIRCUITS AND BOOLEAN ALGEBRA

Logic Gates:

Inputs		Outputs of each gate					
A	B	AND	OR	NAND	NOR	EXOR	EXNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

Boolean Algebra:

Name	AND form	OR form
Identity law	$1A = A$	$0 + A = A$
Null law	$0A = 0$	$1 + A = 1$
Idempotent law	$AA = A$	$A + A = A$
Inverse law	$A\bar{A} = 0$	$A + \bar{A} = 1$
Commutative law	$AB = BA$	$A + B = B + A$
Associative law	$(AB)C = A(BC)$	$(A + B) + C = A + (B + C)$
Distributive law	$A + BC = (A + B)(A + C)$	$A(B + C) = AB + AC$
Absorption law	$A(A + B) = A$	$A + AB = A$
De Morgan's law	$\overline{AB} = \bar{A} + \bar{B}$	$\overline{A + B} = \bar{A}\bar{B}$

SOP & POS: Any Boolean function can be expressed in two different standard forms.

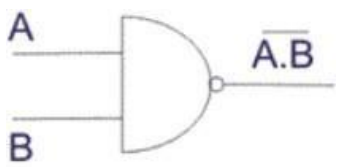
- Boolean algebra is the category of algebra in which the variable's values are the truth values, true and false, ordinarily denoted 1 and 0 respectively. It is used **to analyze and simplify digital circuits or digital gates**.
- We apply Boolean algebra to minimize the number of minterms and maxterms as much as possible

Annulment	Complement	Associative
$A \wedge 0 = 0$	$A \vee \neg A = 1$	$(A \vee B) \vee C = A \vee (B \vee C)$
$A \vee 1 = 1$	$A \wedge \neg A = 0$	$(A \wedge B) \wedge C = A \wedge (B \wedge C)$
Identity	Double Negation	Commutative
$A \wedge 1 = A$	$\neg(\neg A) = A$	$A \vee B = B \vee A$
$A \vee 0 = A$		$A \wedge B = B \wedge A$
Idempotent	De Morgan's	Distributive
$A \vee A = A$	$\neg(A \wedge B) = \neg A \vee \neg B$	$A \wedge (B \vee C) = (A \wedge B) \vee (A \wedge C)$
$A \wedge A = A$	$\neg(A \vee B) = \neg A \wedge \neg B$	$A \vee (B \wedge C) = (A \vee B) \wedge (A \vee C)$
		Absorptive
		$A \vee (A \wedge B) = A$
		$A \wedge (A \vee B) = A$

1. SOP: Sum of Products
2. POS: Products of Sum

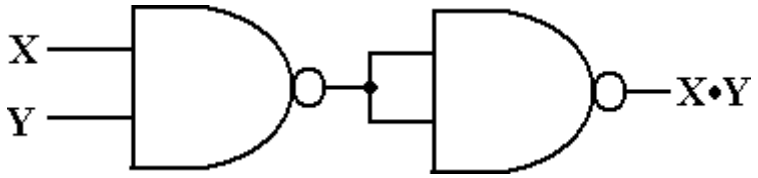
NAND Gate and NOR Gate as Universal Gates:

The first part is an AND gate and the second part is a dot after it represents NOT gate.

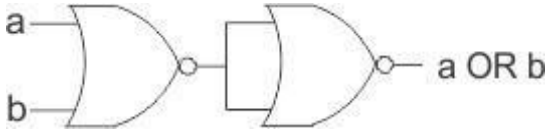


NAND gate

Now we will see the design of an AND gate from NAND gates.



We have seen how the NAND gate can be used to make all the three basic gates by using that alone. Now we will discuss the same in case of NOR gate.



The above diagram is of an OR gate made by only using NOR gates. The output of this gate is exactly like that of a single OR gate. We can see the circuit arrangement of OR gate using, NOR gate is like that of AND gate using NAND gates.

Logic Minimization:

Karnaugh Map

AB \ CD	00	01	11	10
00	0000 (0)	0001 (1)	0011 (3)	0010 (2)
01	0100 (4)	0101 (5)	0111 (7)	0110 (6)
11	1100 (12)	1101 (13)	1111 (15)	1110 (14)
10	1000 (8)	1001 (9)	1011 (11)	1010 (10)

These cells marked with a specific numbers to represent the corresponding minterm or maxterm

Two adjacent cells are differed by only one digit because of the usage of gray code

2 Variable K- Map

A \ B	0	1
0	0	0
1	0	1

AB

3 Variable K- Map

A \ BC	00	01	11	10
0	0	1	1	0
1	0	1	1	1

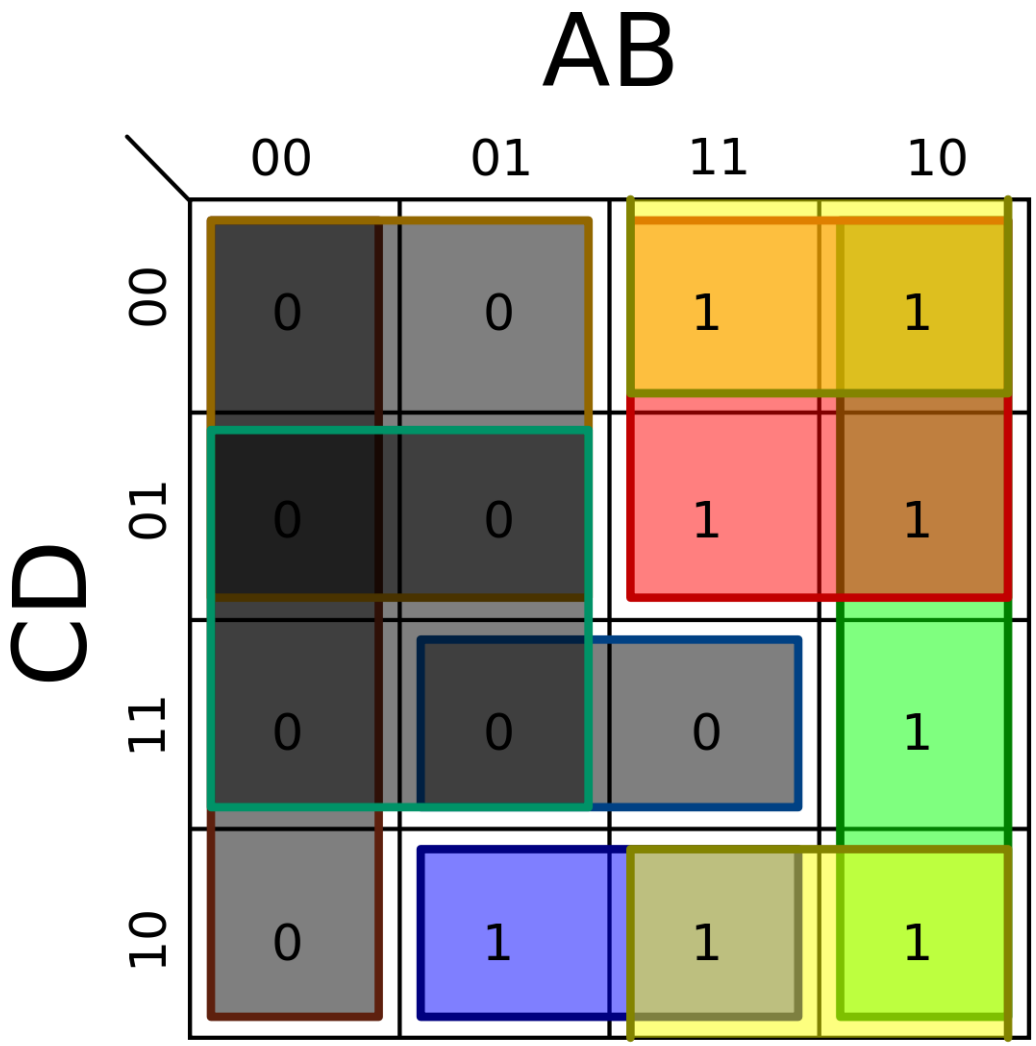
AB + C

4 Variable K- Map

AB \ CD	00	01	11	10
00	0	1	0	0
01	0	1	0	0
11	1	1	1	1
10	0	1	0	0

AB + C'D

Example:



f(A,B,C,D) = E(6,8,9,10,11,12,13,14)

F=AC'+AB'+BCD'+AD'

F=(A+B)(A+C)(B'+C'+D')(A+D')

COMBINATIONAL LOGIC CIRCUITS

Steps to design a combinational logic circuit

- Understand problem description and draw the block diagram with I/P and o/p lines.
- Define the truth table based on functionality.
- Get the simplified expression.
- Implement the logic circuit.

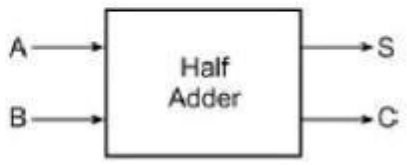
Types of Combinational Circuits:

1. **Half Adder:** Half adder is a combinational logic circuit with two inputs and two outputs. The half adder circuit is designed to add two single bit binary number A and B. It is the basic building block for addition of two single bit numbers. This circuit has two outputs carry and sum

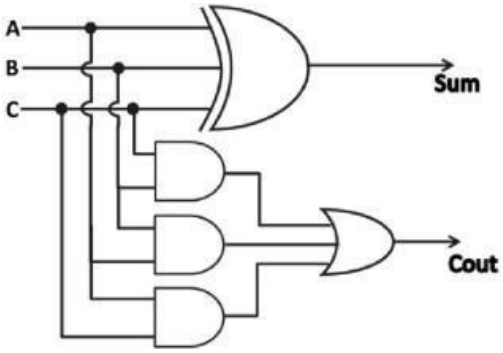
SUM $S = A.\overline{B} + \overline{A}.B$

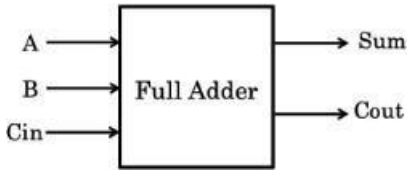
CARRY $C = A.B$

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

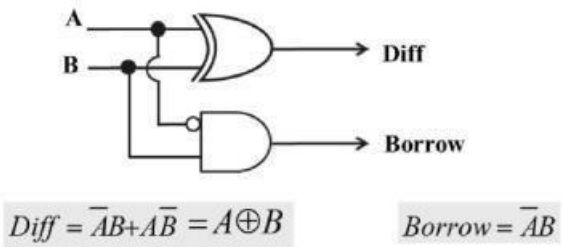


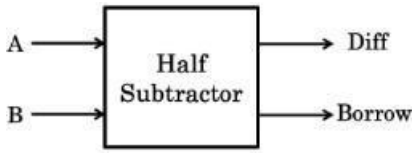
2. **Full Adder:** Full adder is developed to overcome the drawback of Half Adder circuit. It can add two one-bit numbers A and B, and carry c. The full adder is a three input and two output combinational circuit.



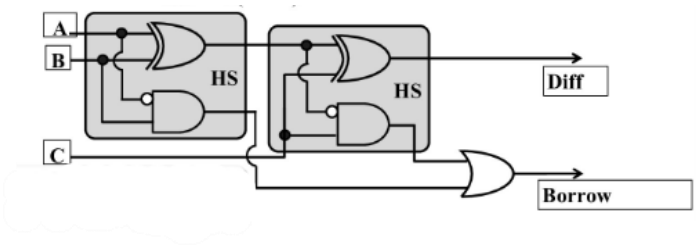


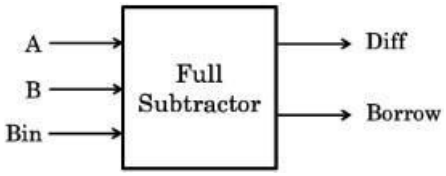
3. **Half Subtractor:** Half subtractor is a combination circuit with two inputs and two outputs (difference and borrow). It produces the difference between the two binary bits at the input and also produces an output (Borrow) to indicate if a 1 has been borrowed.



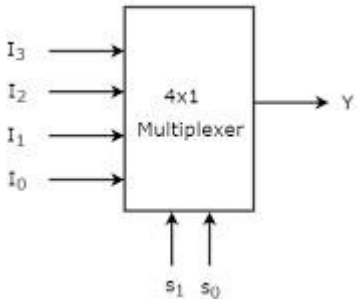


4. **Full Subtractor:** The full subtractor is a combinational circuit with three inputs A,B,Bin and two output Diff and Borrow.

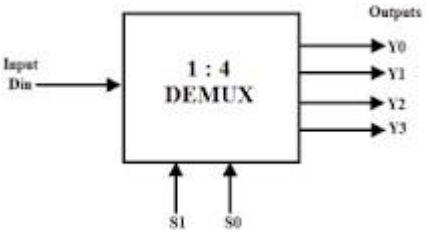




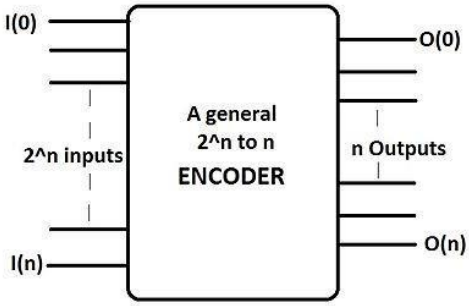
5. **Multiplexer:** Multiplexer is a combinational circuit that has maximum of 2^n data inputs, 'n' **selection lines** and single output line. One of these data inputs will be connected to the output based on the values of selection lines. Since there are 'n' selection lines, there will be 2^n possible combinations of zeros and ones.



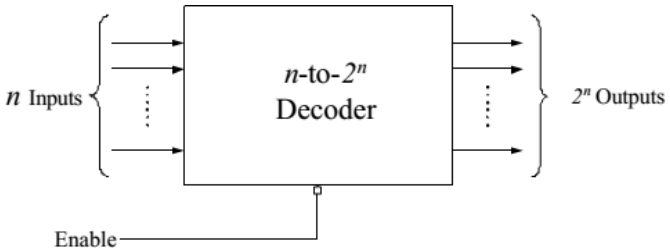
6. **De-Multiplexer:** It consists of 1 input line, 'n' output lines and 'm' select lines. In this, m select lines are required to produce 2^m possible output lines (consider $2^m = n$). For example, a 1-to-4 demultiplexer requires 2 ($2^2 = 4$) select lines to control the 4 output lines.



7. **Encoder:** Encoder is a combinational circuit which is designed to perform the inverse operation of the decoder. An encoder has n number of input lines and m number of output lines. An encoder produces an m bit binary code corresponding to the digital input number. The encoder accepts an n input digital word and converts it into an m bit another digital word.



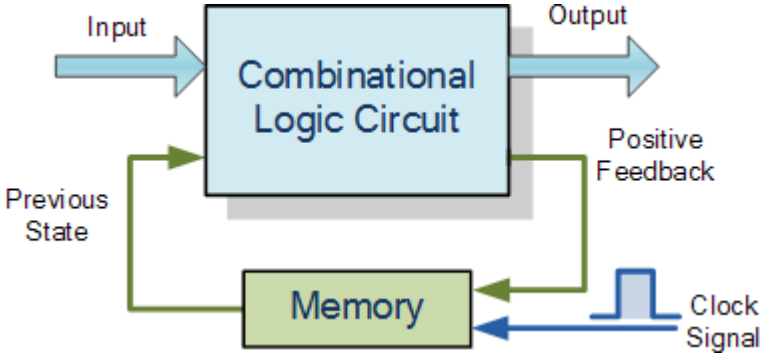
8. **Decoder:** A decoder is a combinational circuit. It has n input and to a maximum m = 2n outputs. Decoder is identical to a demultiplexer without



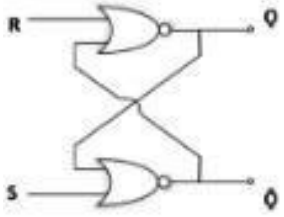
any data input. It performs operations which are exactly opposite to those of an encoder.

SEQUENTIAL LOGIC CIRCUIT

The combinational circuit does not use any memory. Hence the previous state of input does not have any effect on the present state of the circuit. But sequential circuit has memory so output can vary based on input. This type of circuits uses previous input, output, clock, and a memory element.

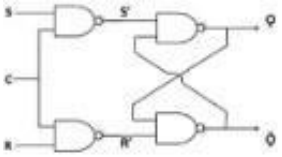


1. **Latches:** Latch is an electronic device, which changes its output immediately based on the applied input. It is used to store either 1 or 0 at any specified time. It consists of two inputs namely "SET" and RESET and two outputs, which are complement to each other.
   a. **RS Latch:**

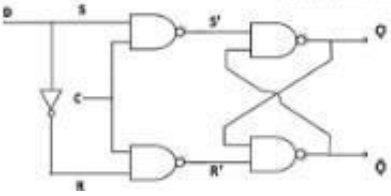


R	S	Q	Q'	State
0	1	1	0	Set
0	0	1	0	No Change
1	0	0	1	Reset
0	0	0	1	No Change
1	1	0	0	Invalid
0	0	-	-	RACE

   b. **D Latch:**



C	S	R	S'	R'	Q	Q'	State
0	X	X	1	1			No Change
1	0	0	1	1			No Change
1	0	1	1	0	0	1	Reset
1	1	0	0	1	1	0	Set
1	1	1	0	0	-	-	Invalid



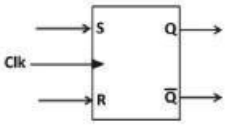
C	D	Q	Q'	State
0	X			No Change
1	0	0	1	Reset
1	1	1	0	Set

2. **Flip Flop:** Flip-flop is a basic digital memory circuit, which stores one bit of information. Flip flops are the fundamental blocks of most sequential circuits. It is also known as a bistable multivibrator or a binary or one-bit memory.

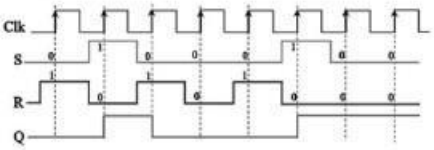
   a. **S-R Flip Flop:**

Truth Table

Clk	S	R	Q_+	State
↓	X	X	Q	No Change
↑	0	0	Q	No Change
	0	1	0	Reset
	1	0	1	Set
	1	1	-	Invalid



Wave forms



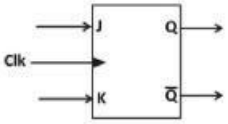
Excitation Table

Q	Q_+	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

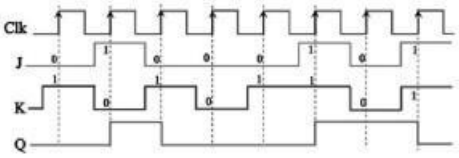
   b. **D Flip Flop:**

Truth Table

Clk	J	K	Q_+	State
↓	X	X	Q	No Change
↑	0	0	Q	No Change
	0	1	0	Reset
	1	0	1	Set
	1	1	Q'	Toggle



Wave forms



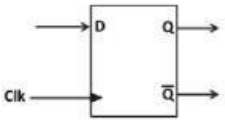
Excitation Table

Q	Q_+	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

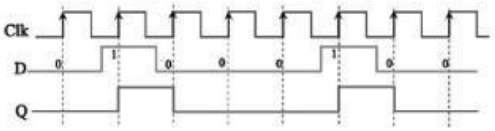
   c. **JK Flip Flop:**

Truth Table

Clk	D	Q_+	State
↓	X	Q	No Change
↑	0	0	Reset
	1	1	Set



Wave forms



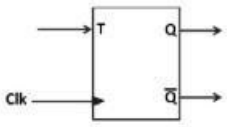
Excitation Table

Q	Q_+	D
0	0	0
0	1	1
1	0	0
1	1	1

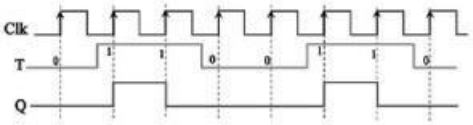
d. T Flip Flop:

Truth Table

Clk	T	Q_+	State
↓	X	X	No Change
↑	0	Q	No Change
	1	Q`	Toggle



Wave forms



Excitation Table

Q	Q_+	T
0	0	0
0	1	1
1	0	1
1	1	0

3. **Registers:** It is a group of flip-flops. Its basic function is to hold information within a digital system so as to make it available to the logic units during the computing process. However, a register may also have additional capabilities associated with it.

   Types:

   a. **Serial Shift Register**
   b. **Parallel Shift Register**
   c. **SISO Shift Register**
   d. **SIPO Shift Register**
   e. **PISO Shift Register**
   f. **PIPO Shift Register**

4. **Counters:** It is essentially a register that goes through a predetermined sequence of states. The gates in the counter are connected in such a way as to produce the prescribed sequence of binary states.

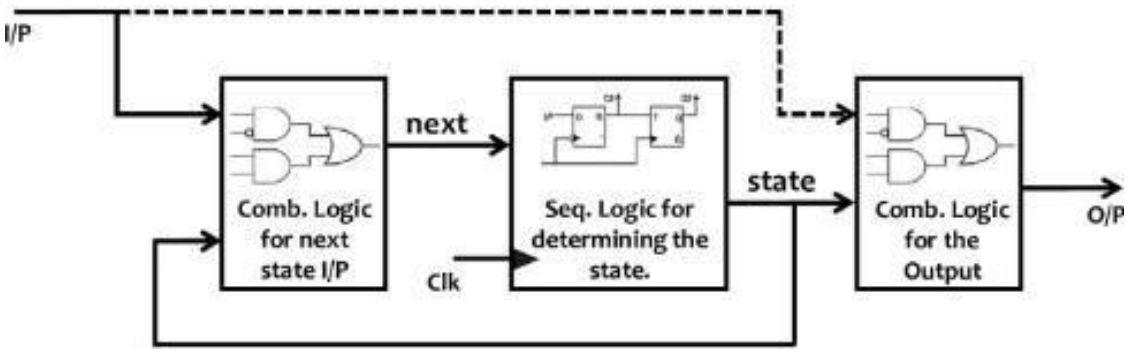
   Types:

   a. **Asynchronous Counter**
   b. **UP/DOWN Counter**
   c. **Ripple Counter**
   d. **Ring Counter**

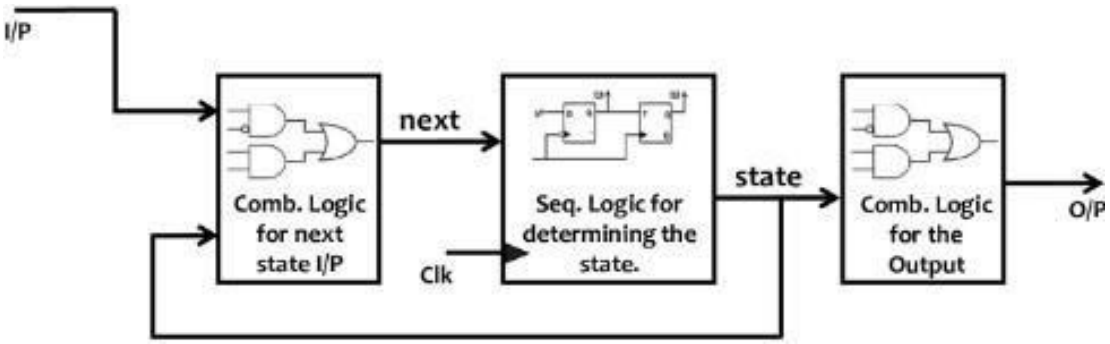
FINITE STATE MACHINE

Finite State Machines (FSMs) are at the heart of most digital design. The basic idea of an FSM is to store a sequence of different unique states and transition between them depending on the values of the inputs and the current state of the machine. The FSM can be of two types: Moore (where the output of the state machine is purely dependent on the state variables) and Mealy (where the output can depend on the current state variable values AND the input values).

1. Mealy Machines:



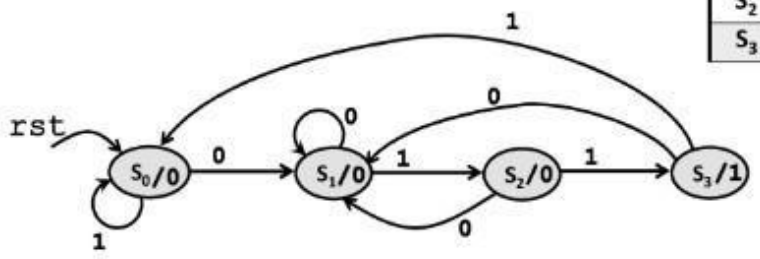
2. Moore Machines:



3. Sequence Detector Moore:

Sequence : 110

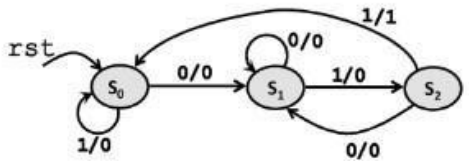
State	Meaning
S_0	Initial state
S_1	0
S_2	10
S_3	110



4. Sequence Detector Mealy:

Sequence : 110

State	Meaning
S_0	Initial state
S_1	0
S_2	10



Moore Machine	Mealy Machine
Output depends only upon the present state.	Output depends on the present state as well as present input.
If input changes, the output does not change.	If input changes, output also changes.
More states are required.	Less number of states are required.
There is less hardware requirement for circuit implementation.	There is more hardware requirement for circuit implementation.
They react slower to inputs(One clock cycle later).	They react faster to inputs.
Synchronous output and state generation.	Asynchronous output generation.
Output is placed on states.	Output is placed on transitions.
Easy to design.	It is difficult to design.

Glitches & Hazards

Glitch: A glitch is a fast "spike" usually unwanted.

Hazard: A hazard in a circuit may produce a glitch. if the propagation delays are unbalanced.

Types of Hazards:

Static 1 - Hazard

- Possibility of a circuit's output producing a 0 glitch when we would expect the output to remain at a steady 1.

Static 0 - Hazard

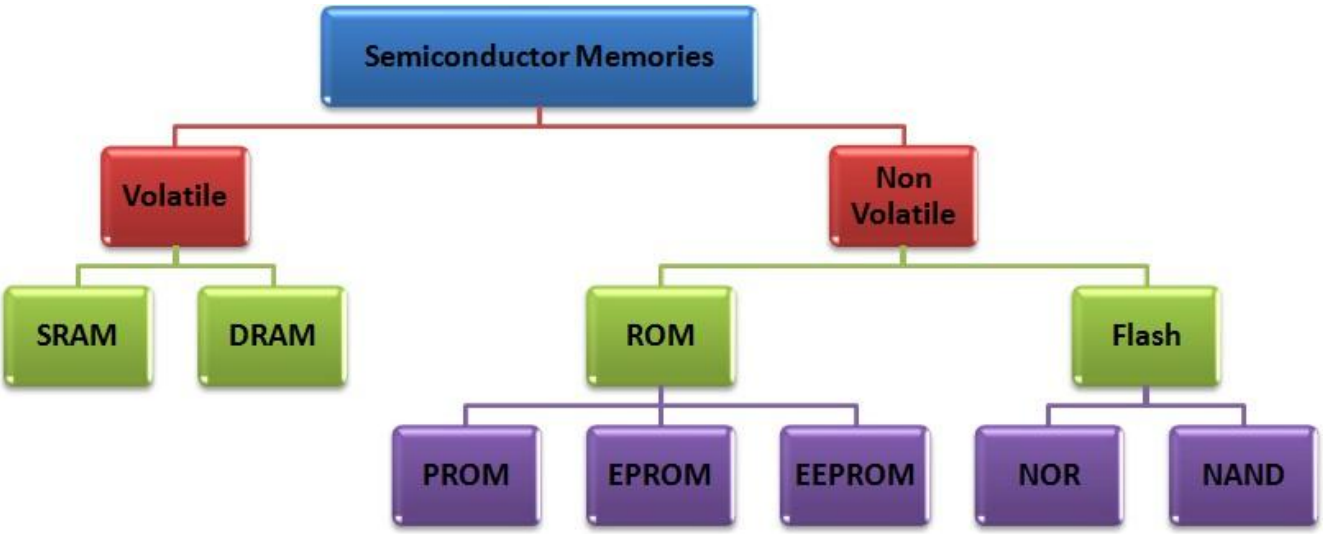
- Possibility of a circuit's output producing a 1 glitch when we would expect the output to remain at a steady 0.

Dynamic Hazard

- Possibility of the output of a circuit changing more number of times when it should change from (0 to 1) or (1 to 0).

MEMORIES

Memories are one of the most useful VLSI building blocks. One reason for their utility is that memory arrays can be extremely dense. A memory unit is a collection of storage cells with associated circuits needed totransfer information in and out of the device. The binary information is transferred for storage and from which information is available when needed forprocessing. When data processing takes place, information from the memory is transferred to selected registers in the processing unit. Intermediate and final results obtained in the processing unit are transferred back to be stored in memory.



VERILOG HDL

Verilog is a Hardware Description Language (HDL). It is a language used for describing a digital system such as a network switch, a microprocessor, a memory, or a flip-flop. We can describe any digital hardware by using HDL at any level. Designs described in HDL are independent of technology, very easy for designing and debugging, and are normally more useful than schematics, particularly for large circuits.

Verilog Abstraction Levels

- Verilog supports 4 levels of abstraction namely,
- ***Switch level***-Module is implemented in terms of switches, we can represent the entire circuit as a CMOS circuit.
- ***Gate level***-Module is implemented in terms of gates, which is the lowest level of abstraction. Basic logic gates are available as predefined primitives. In the digital library of Verilog, these logic gates are already saved and can be used directly like and, or, xor, nand, nor, etc.
- ***Data flow level***- At this level, the module is designed by specifying the data flow, like how the data is flowing in the circuit based on that the equation should be written. Signals are assigned by the data manipulating equations, design is implemented using continuous assignment i.e. assign statement is used in this level, the assignments present in it are concurrent in nature.
- ***Behavioral level***- This is the highest level of abstraction provided by HDL, this level describes the behavior of the system. Different elements like functions, tasks, and blocks can be used, two important constructors are initial and always.

CONCLUSION

I have completed the given online VLSI design methodologies course in 30-25 active days .I am very thankful to Maven Silicon for offering me this course as a part of my online Internship course, from which I have learnt a variety of new concepts and have also developed my understanding of already known concept.

I have now gained a good idea especially on the front end section of the VLSI course which imparts the VLSI Design Flow, Digital Design and RTL programming using Verilog HDL, and thus,in future, I am keen to learn the further concepts that follow the stages of VLSI that I have learnt so far which involves the back end design flow.

FUTURE SCOPE

VLSI has always been and will remain one of the most important fields. With the automation industry going all electric, and the world going towards a more technology-based environment, VLSI will always find a place here or there. The industry has started slowing down. This is because of the physical limitation of silicon technology. We as researchers have not yet found a material that can replace silicon or have not yet perfected Quantum computing. There is a renewed interest where people have started concentrating on merging VLSI + Computer architecture and system designs. The performance improvement from Moore's law scaling is providing diminishing returns in terms of cost and yield. Therefore, we can conclude that the industry has a lot of scope still left in it.

References

https://elearn.maven-silicon.com/material/dashboard/4